

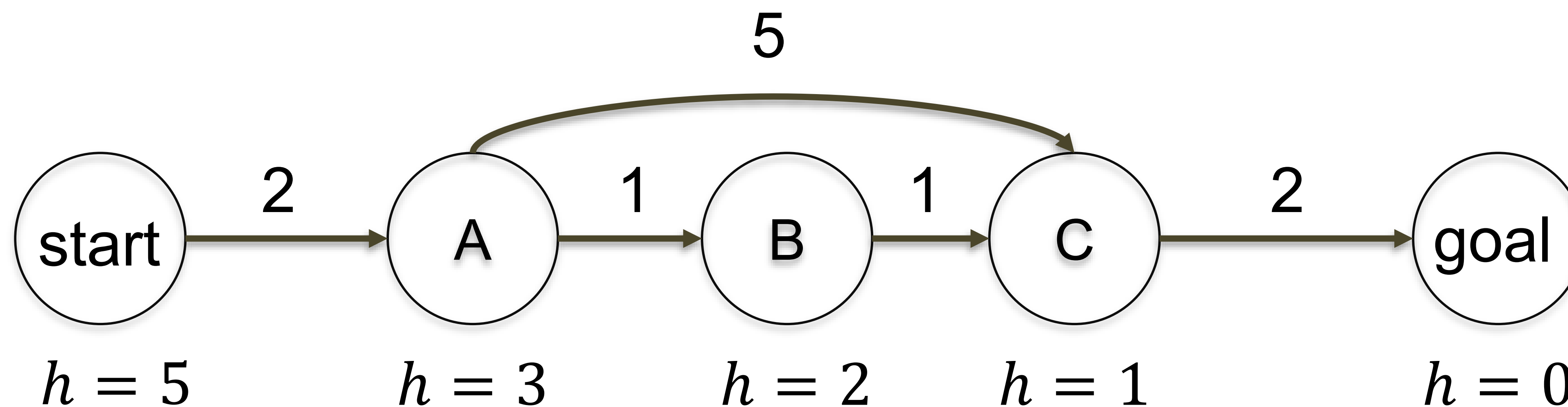
Recap: Greedy best-first search

An informed search algorithm that expands the node that *appears to be closest to the goal*, as estimated by a heuristic function $h(n)$ (Informed Search)

$h(n)$: Estimated cost of the cheapest path from the current node n to a goal. (for goal node: $h(n) = 0$)

Greedy best-first search *ignores the actual path cost*

Greedy best-first search is not optimal



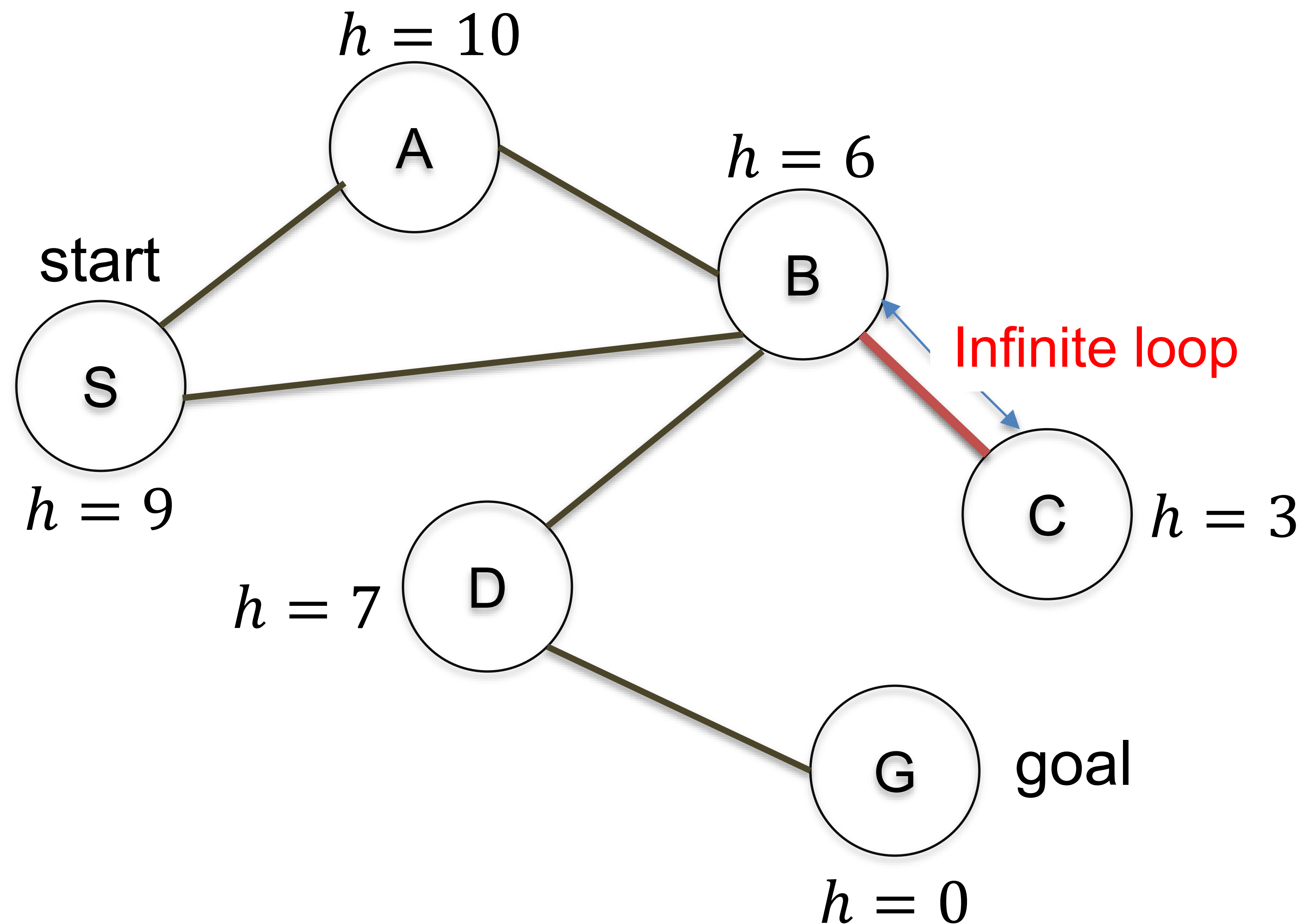
Greedy best-first search: start $\rightarrow$ A $\rightarrow$ C $\rightarrow$ goal, path cost = 9

Actual optimal path: start $\rightarrow$ A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ goal, path cost = 6

Greedy best-first search uses only the heuristic to choose the node that seems closest to the goal, ignoring the actual cost from the start—so it can miss the cheapest path.

Greedy best-first search is not complete

Use greedy best-first search to find a path from the start to the goal, assuming a tree search strategy.



Each node is labeled with its heuristic estimate $h(n)$ (shown beside the node),

Start at S:

- The search begins at node **S** ($h=9$).
- Neighbors: A ($h=10$) and B ($h=6$), added to the frontier.
- Frontier: [**B($h=6$)**, **A($h=10$)**].

Expand B:

- B** has the lowest heuristic value (6) in the frontier, so it's expanded next.
- Neighbors: A ($h=10$), S ($h=9$), C ($h=3$), and D ($h=7$), added to the frontier
- Frontier: [**C($h=3$)**, **D($h=7$)**, **S($h=9$)**, **A($h=10$)**].

Expand C:

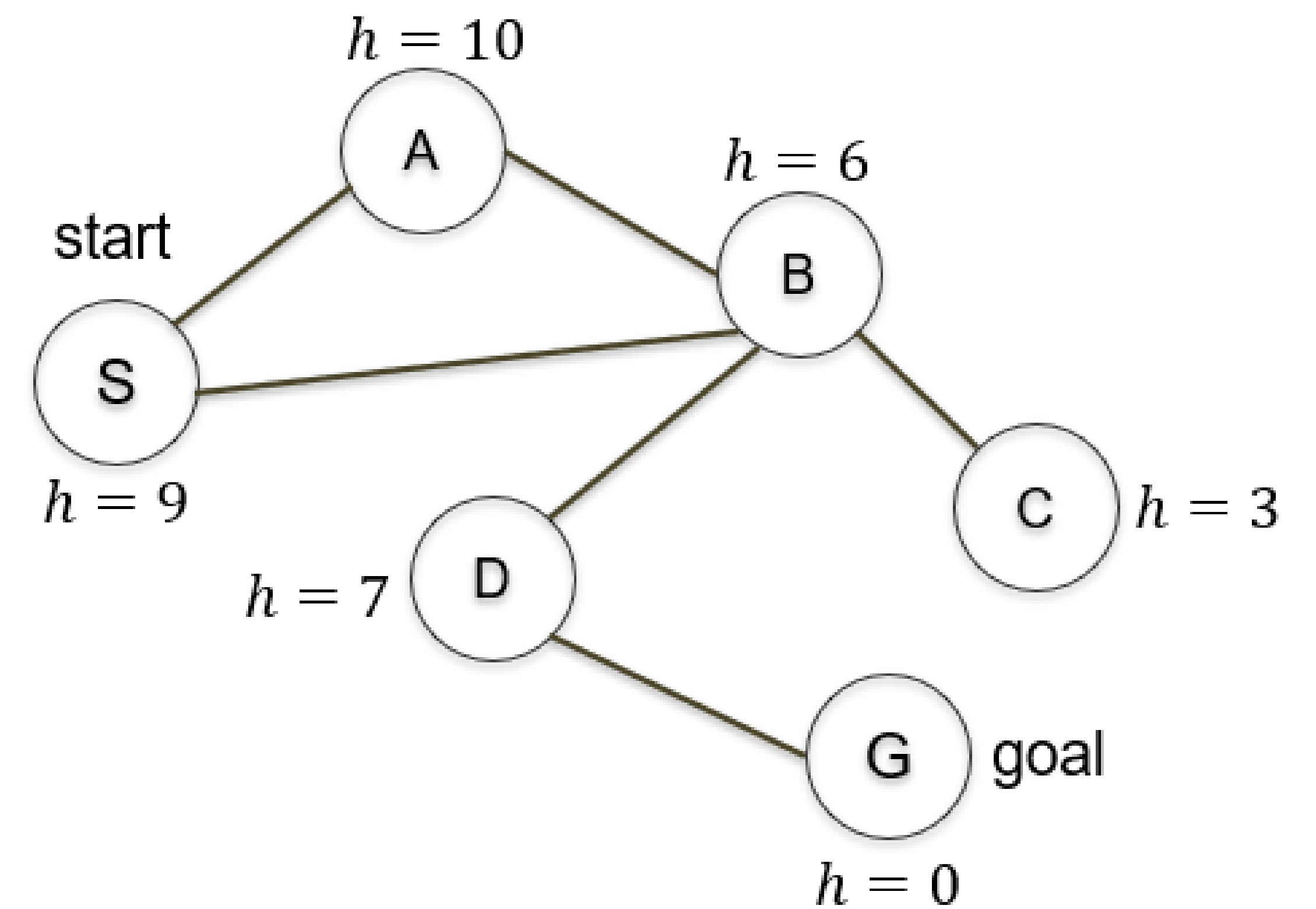
- C** has the lowest heuristic value (3) and is chosen next.
- Neighbor: B ($h=6$), added back to the frontier.
- Frontier: [**B($h=6$)**, **D($h=7$)**, **S($h=9$)**, **A($h=10$)**].

Expand B:

- B** again has the lowest heuristic value (6) and is expanded. The path is currently **S -> B -> C -> B**.
- B's neighbors: (C, D, S, A) are again added to the frontier.
- Frontier: [**C($h=3$)**, **D($h=7$)**, **S($h=9$)**, **A($h=10$)**].

Expand C:

- C** has the lowest heuristic value (3) and is chosen next.
- C's only neighbor is B ($h=6$).
- B is added back to the frontier.
- Frontier: [**B($h=6$)**, **D($h=7$)**, **S($h=9$)**, **A($h=10$)**].

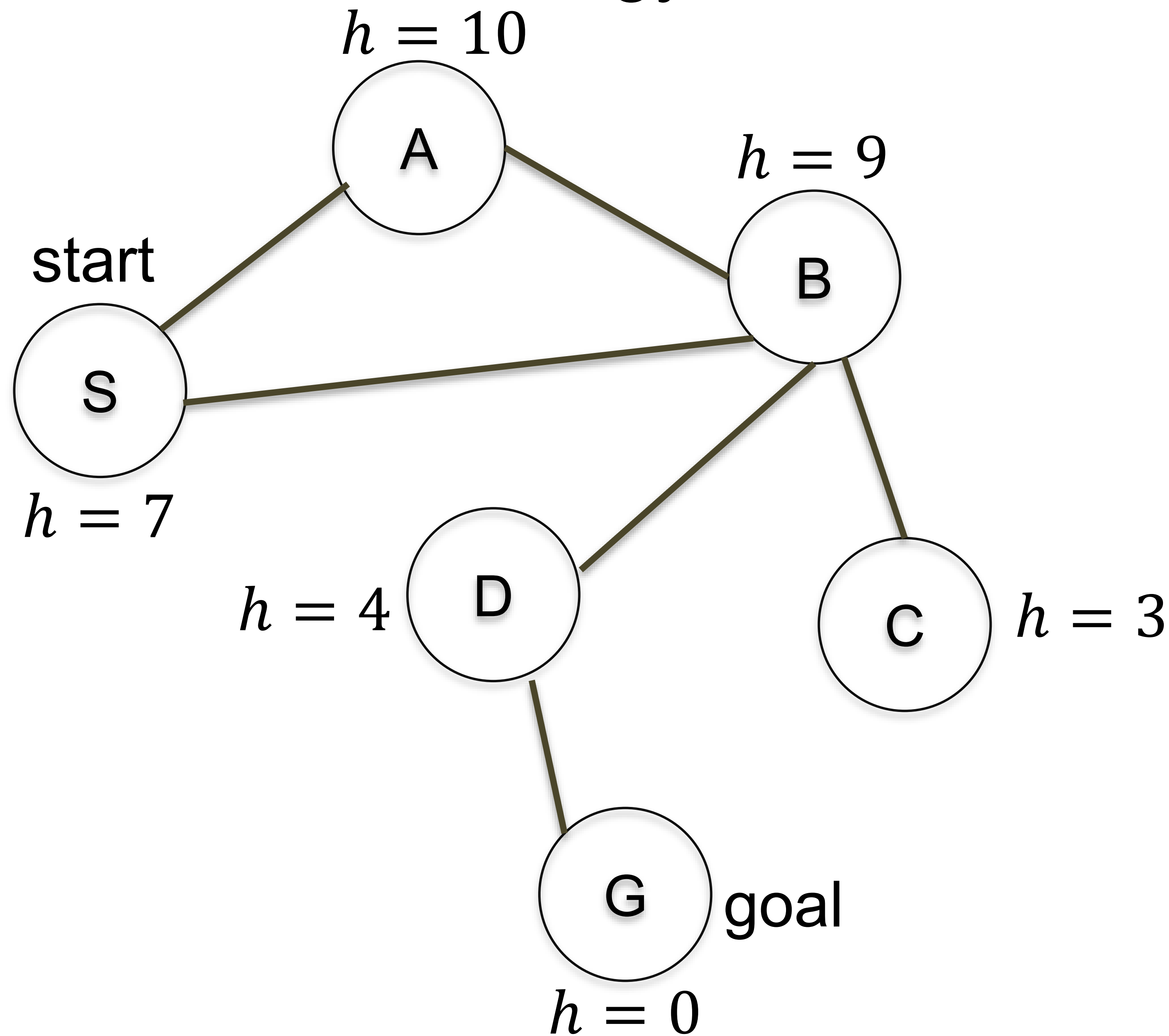


Infinite loop

...

Earlier example:

Which path does greedy best-first search return, assuming tree search strategy?



Each node is labeled with its heuristic estimate $h(n)$ (shown beside the node),

Start at S (h=7):

- Neighbors: A (h=10), B (h=9).
- Frontier: [**B(h=9)**, **A(h=10)**].

Expand B (h=9):

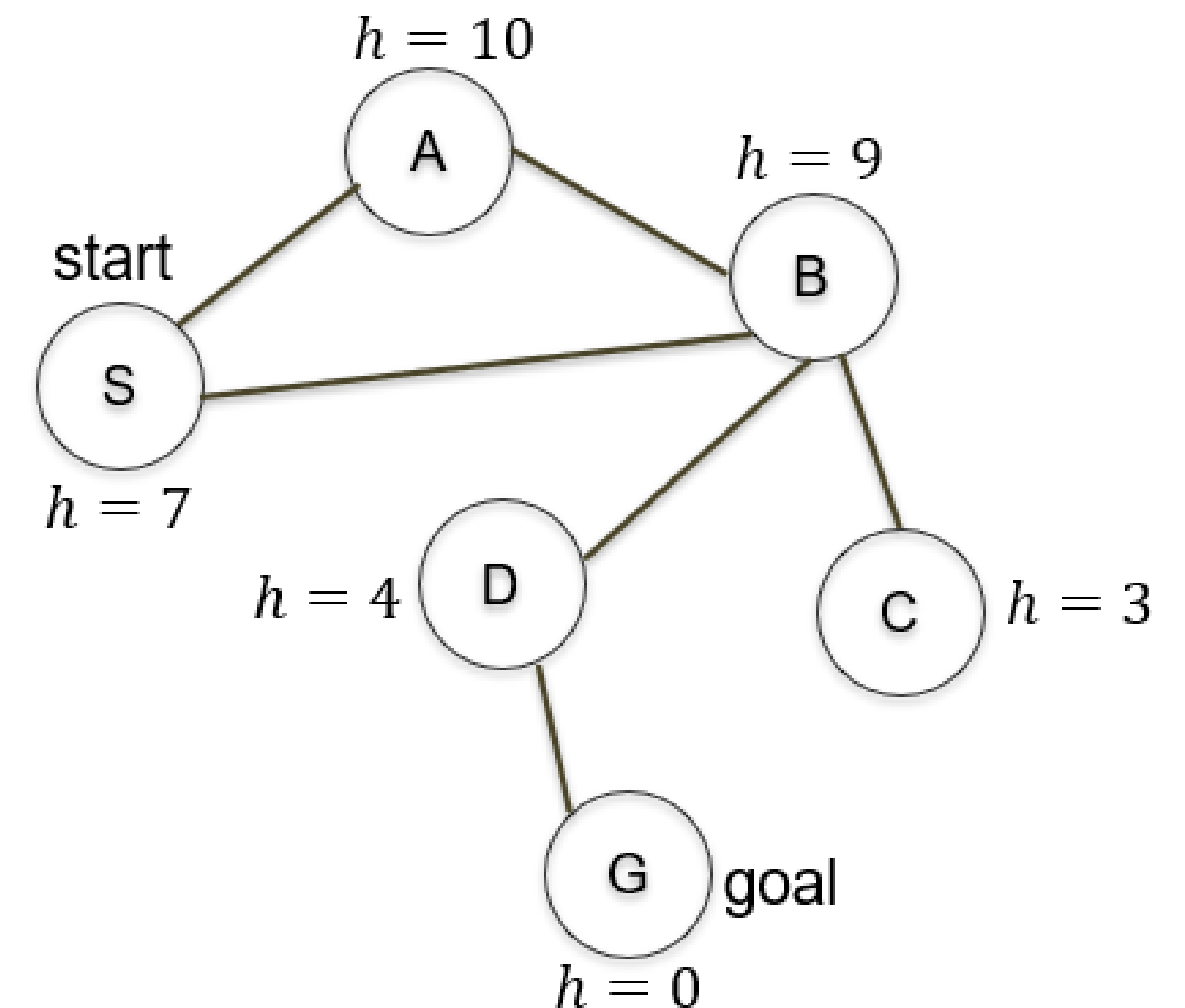
- Neighbors: S (h=7), A (h=10), C (h=3), D (h=4).
- Frontier: [**C(h=3)**, **D(h=4)**, **S(h=7)**, **A(h=10)**].

Expand C (h=3):

- Neighbors: B (h=9).
- Frontier: [**D(h=4)**, **S(h=7)**, **B(h=9)**, **A(h=10)**].
- The path so far is **S -> B -> C**.

Expand D (h=4):

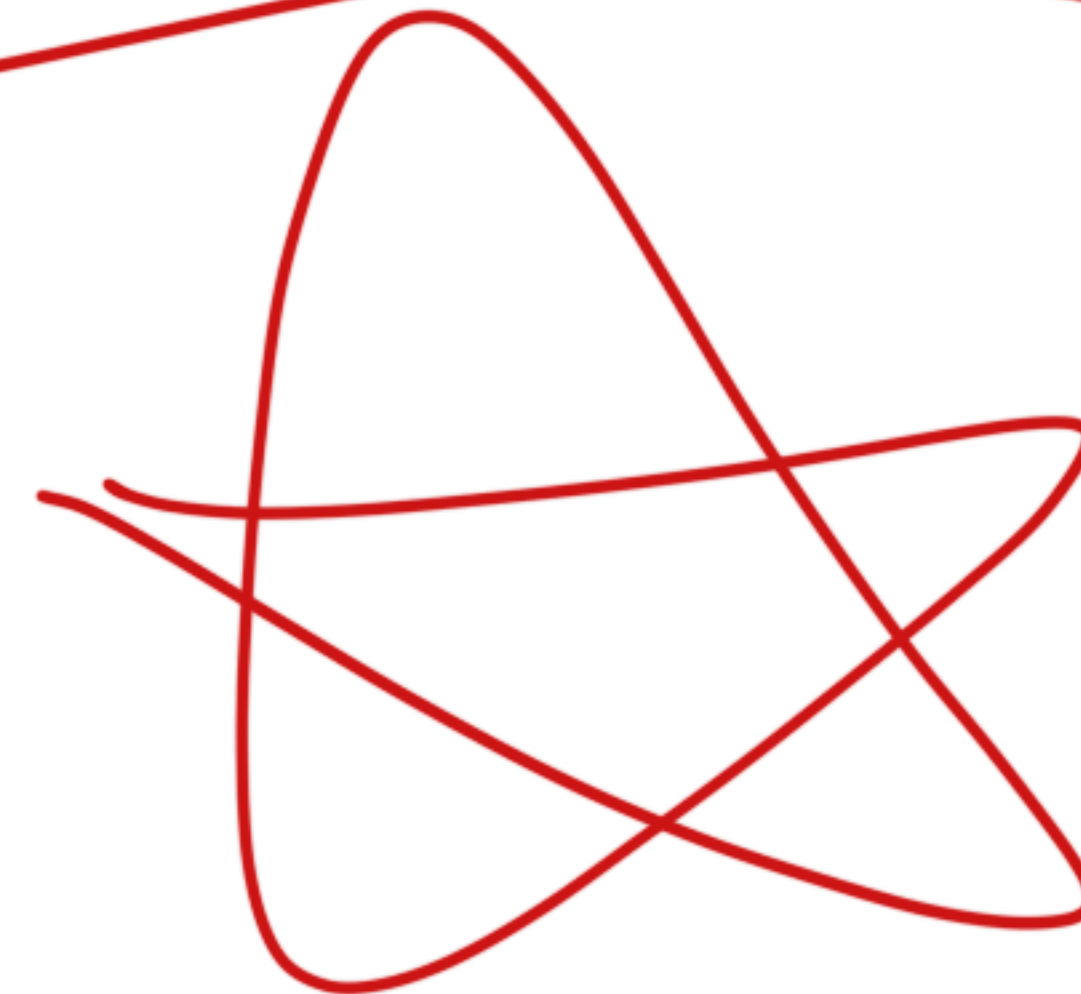
- The algorithm now chooses **D** because it has the lowest heuristic value (4) in the frontier.
- D's neighbors are B (h=9) and G (h=0).
- The goal state, **G**, has a heuristic of 0. The algorithm chooses G next.
- The path found is **S -> B -> D -> G** (this is because we previously get to D from B, that is, D's predecessor is B)



Properties of greedy best-first search

Complete? **No**—can get stuck in loops

Optimal? **No**—not guaranteed to render lowest cost solution.



Completeness: Does the algorithm always find a solution if one exists?

Optimality: Does it find the lowest-cost (best) solution among all possible ones?

Properties of Dijkstra's algorithm (UCS)



Complete? **Yes** –when all edge costs are non-negative.

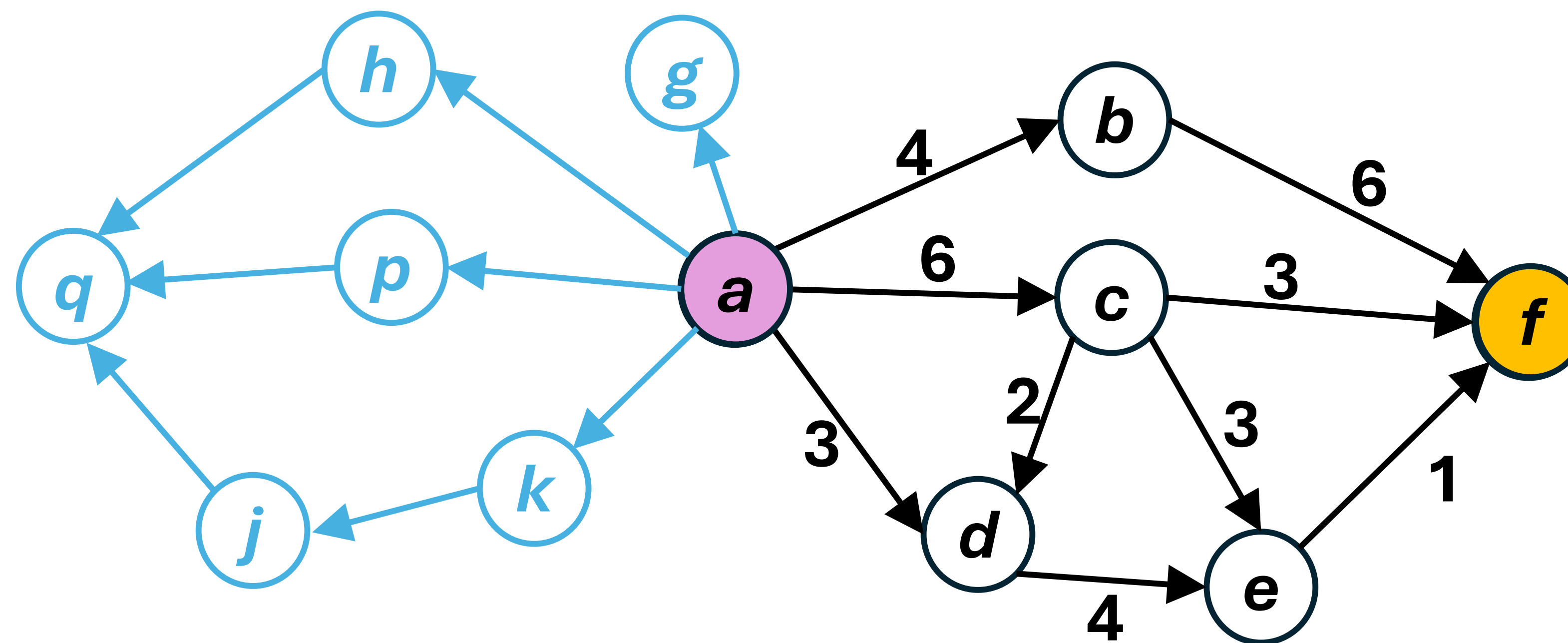
Optimal? **Yes** –when all edge costs are non-negative.

Completeness: Does the algorithm always find a solution if one exists?

Optimality: Does it find the lowest-cost (best) solution among all possible ones?

Dijkstra's algorithm may be inefficient

If the starting node a has neighbors in the opposite direction of the target vertex f , will Dijkstra's algorithm explore them? If so, how can we improve its efficiency?



A*: Where *heuristics* meet *reality*

It evaluates nodes by combining $g(n)$, the actual cost to reach the node, and $h(n)$, the estimated cost to get from the node to the goal.

$$f(n) = g(n) + h(n) \quad \leftarrow f(n): \text{Estimated total cost of the cheapest path from the start to the goal via node } n$$



$g(n)$: Actual cost of the path from the start node to the current node n . (for start node: $g(n) = 0$)

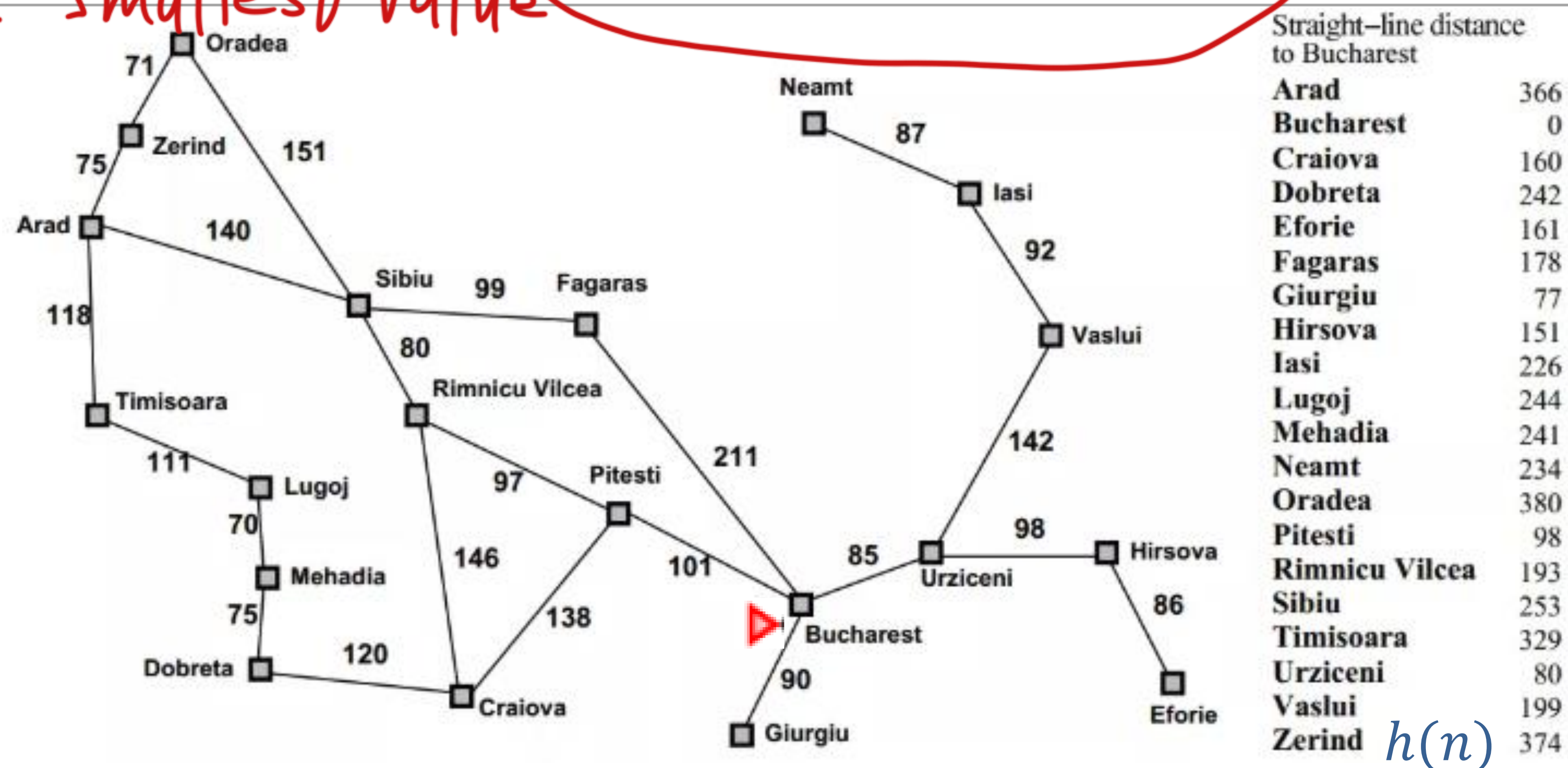
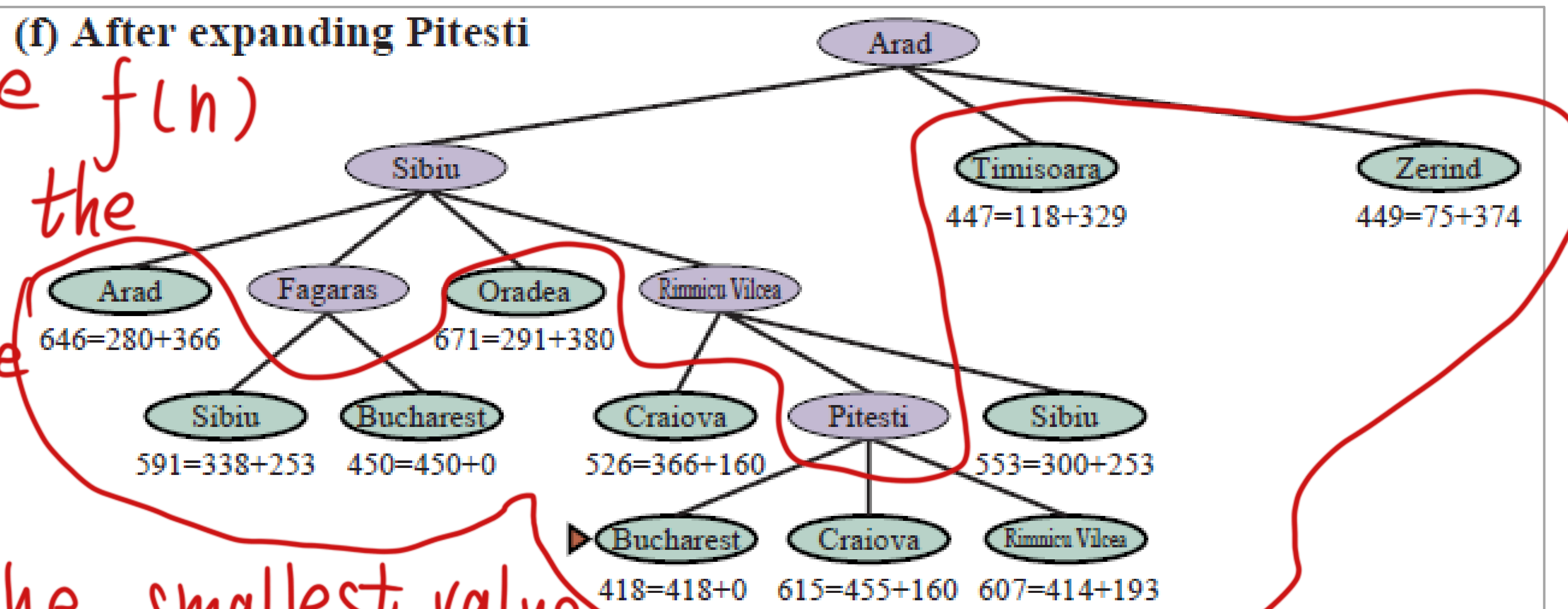
$$f(n) = g(n) + h(n)$$

$h(n)$: Estimated cost of the cheapest path from the current node n to the goal. (for goal node: $h(n) = 0$)

Example: Path planning with A*

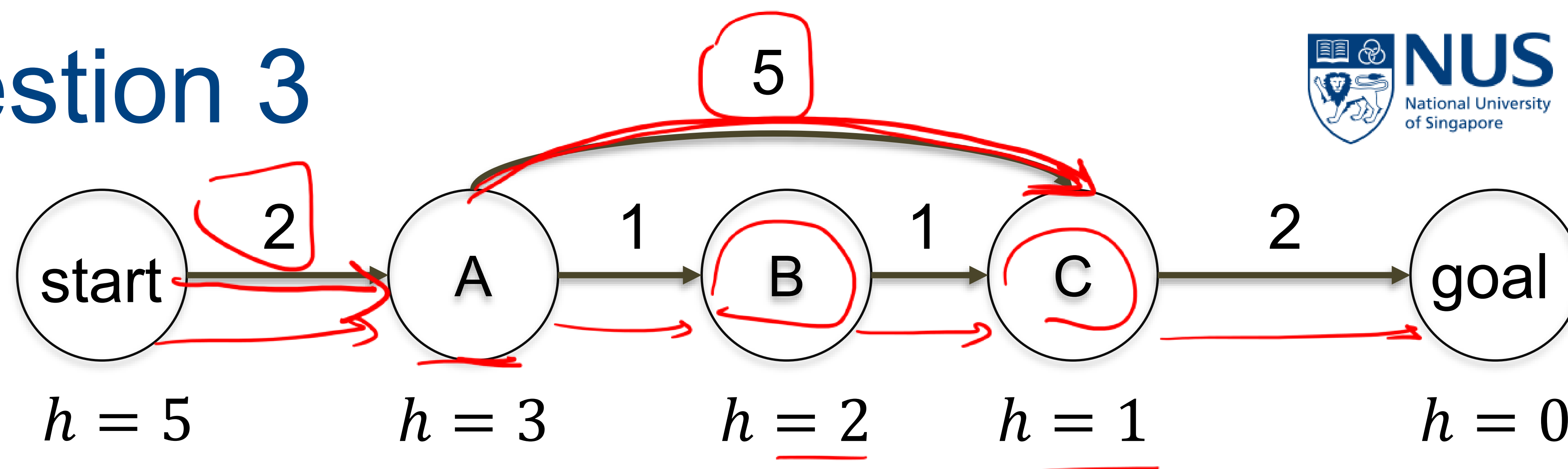
Compare the $f(n)$
among all the
nodes in the
frontier.

Then choose the smallest value



Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P105

Quick question 3



Start at start (f=5):

- Neighbors: A (f=5)
- Frontier: [A(f=5)].

Expand A (f=5):

- Neighbors: B (f=5), C (f=8)
- Frontier: [B(f=5), C(f=8)].

Expand B (f=5):

- Neighbors: C (f=5).
- Frontier: [C(f=5)]
- The path so far is **start -> A -> B -> C**.

Expand C (f=5):

- C's only neighbor is **goal** (h=0).
- The goal state, **goal**, has a heuristic of 0. The algorithm chooses goal next.
- The path found is **start -> A -> B -> C -> goal**, which is the optimal cost

$$f(B) = g(B) + h(B)$$

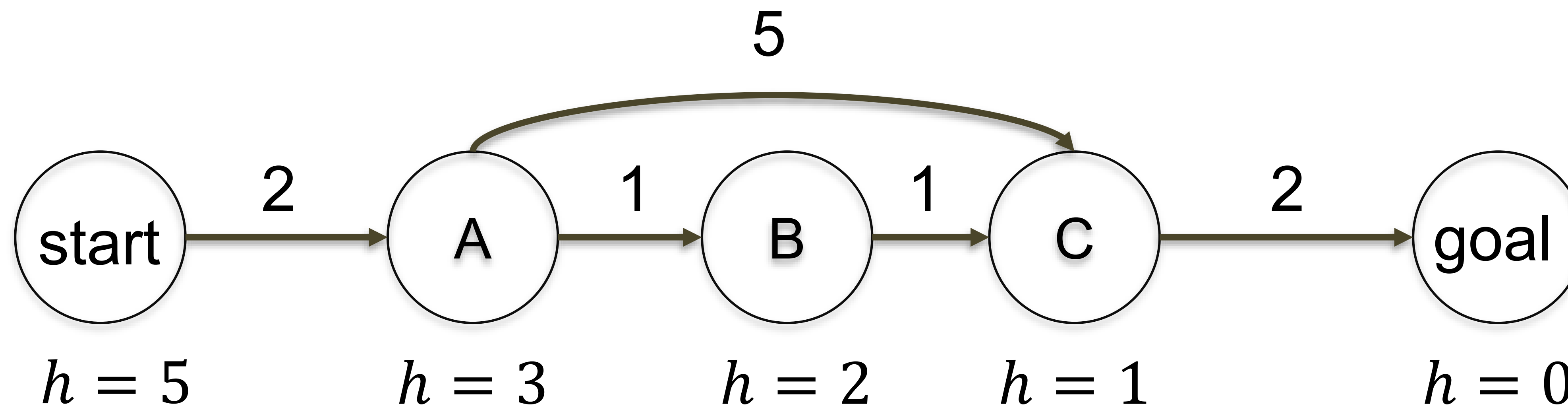
$$2 + 1 + 2$$

$$f(C) = g(C) + h(C)$$

$$= 2 + 5 + 1$$

$$= 8$$

Compare with greedy best-first search:



Greedy best-first search: $\text{start} \rightarrow \text{A} \rightarrow \text{C} \rightarrow \text{goal}$, path cost = 9

A* search: $\text{start} \rightarrow \text{A} \rightarrow \text{B} \rightarrow \text{C} \rightarrow \text{goal}$, path cost = 6, which is the optimal cost

*Greedy best-first search uses **only the heuristic** to pick the node that seems closest to the goal, ignoring the actual cost from the start. In contrast, A* search **considers both the actual path cost and the heuristic**.*

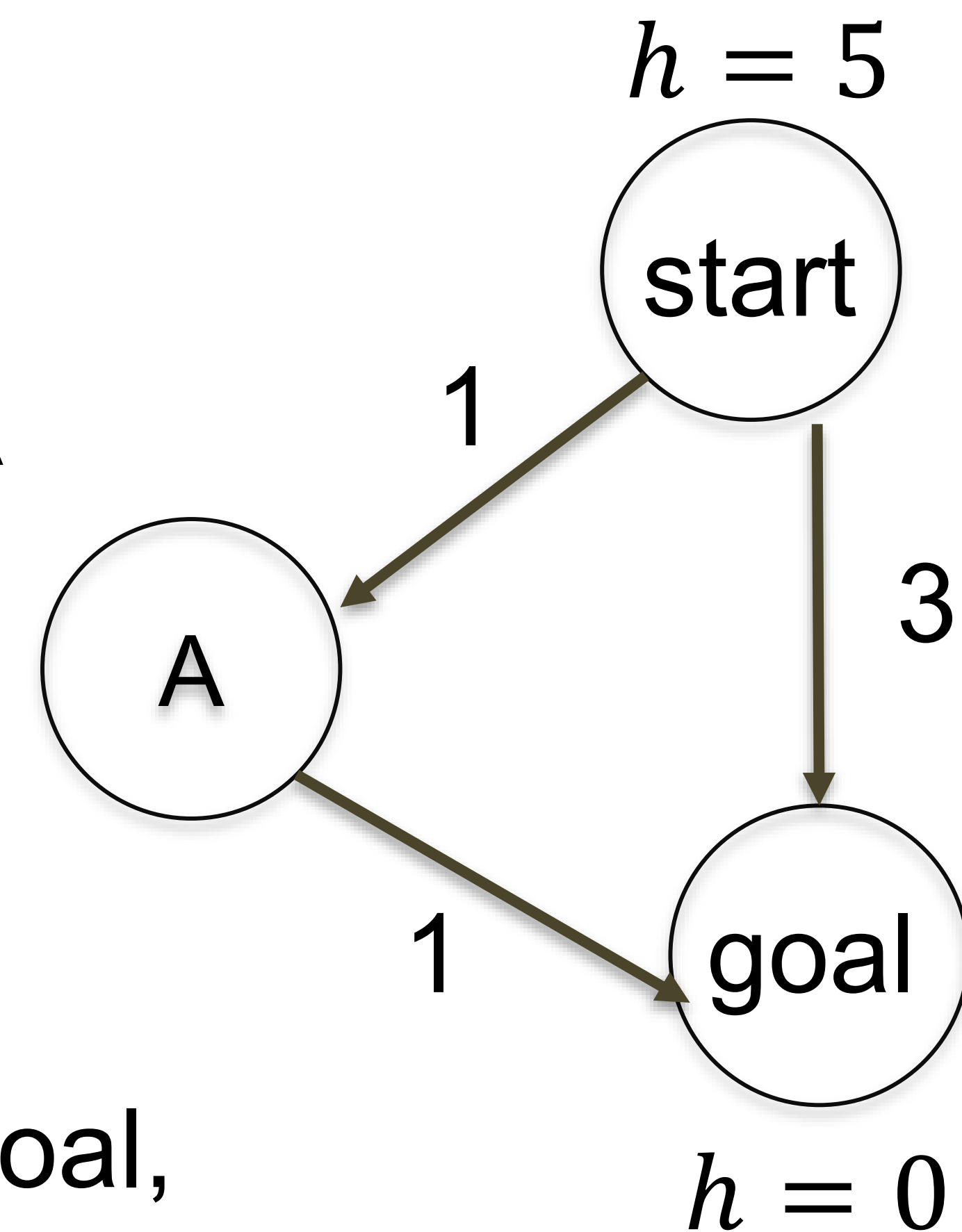
Quick question 4

Given $h(A) = 1$, A^* chooses the path $\text{start} \rightarrow A \rightarrow \text{goal}$, resulting in a total cost of 2

Given $h(A) = 2$, A^* chooses either the path $\text{start} \rightarrow A \rightarrow \text{goal}$ or $\text{start} \rightarrow \text{goal}$

Given $h(A) = 3$, A^* chooses the path $\text{start} \rightarrow \text{goal}$, resulting in a total cost of 3

Optimal solution: $\text{start} \rightarrow A \rightarrow \text{goal}$, total cost = 2



Can we put conditions on the choice of heuristic to guarantee optimality?

Admissible heuristics

A heuristic $h(n)$ is **admissible** if for every node n , $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost to reach the goal state from n .

Admissible heuristics are optimistic: they **never overestimate** the cost to reach the goal, always estimating it as less than or equal to the actual cost.

Example:

Driving from City A to B

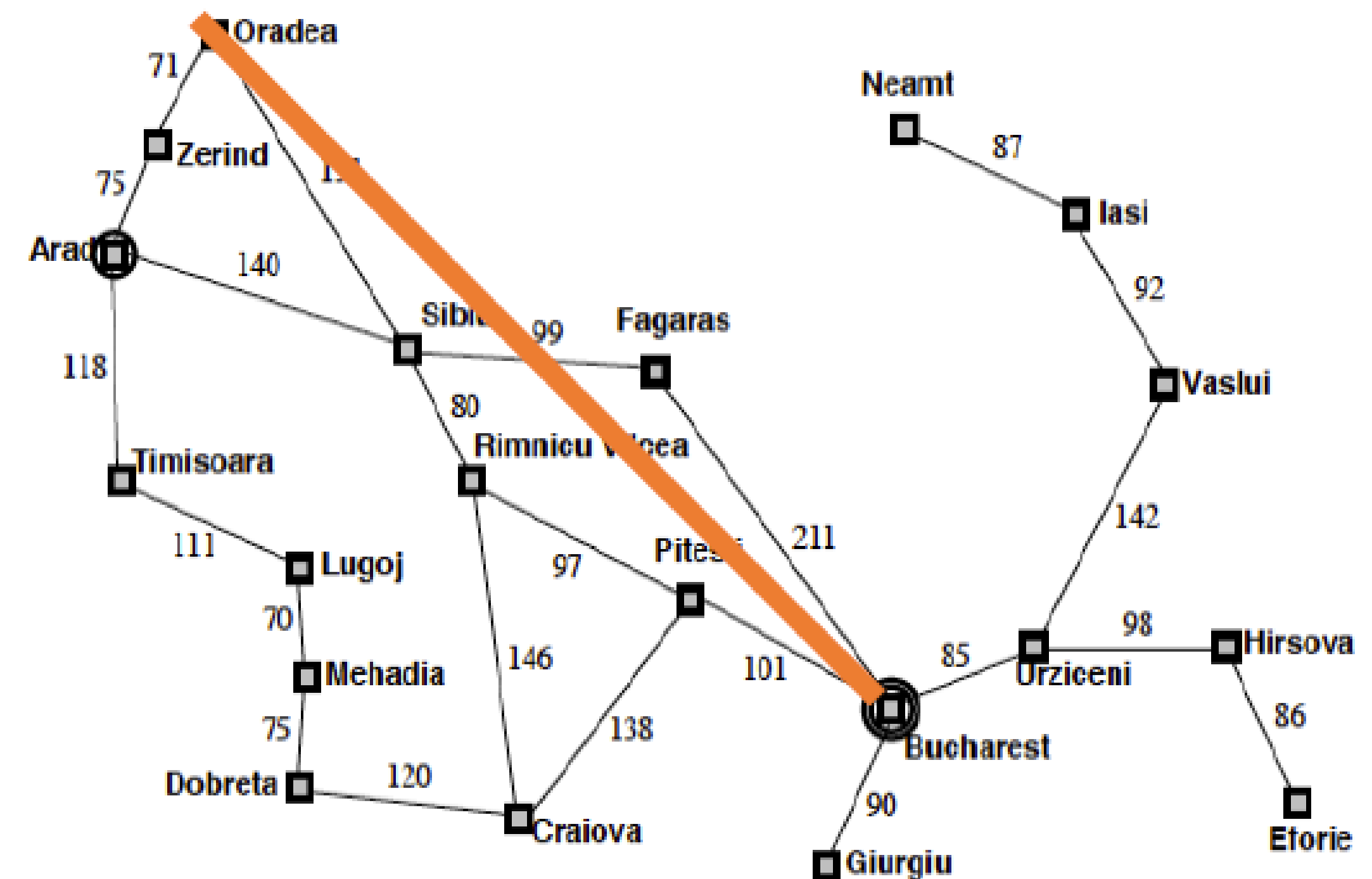
- Actual road distance: 100 km
- Straight-line (Euclidean) distance as heuristic: 80 km

The heuristic *never overestimates* → **Admissible**

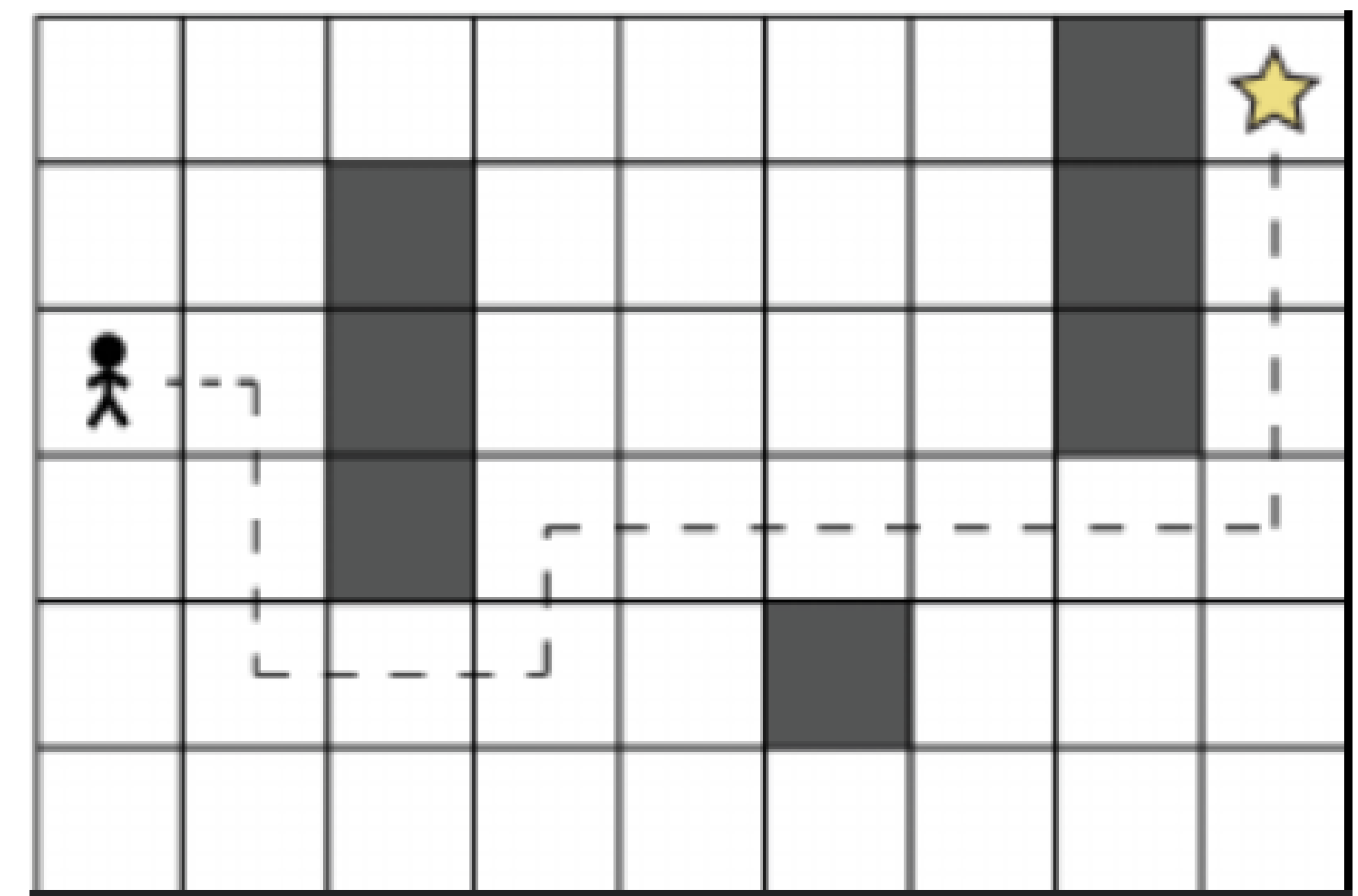


Examples of admissible heuristics

In navigation, straight line (Euclidean) distance -- $h_{ED}(n)$, **never overestimates** the actual road distance



In grid worlds with 4-directional moves, Manhattan distance -- $h_{MD}(n)$, **never overestimates** the actual cost (**actual path $\geq$ sum of horizontal + vertical steps**)

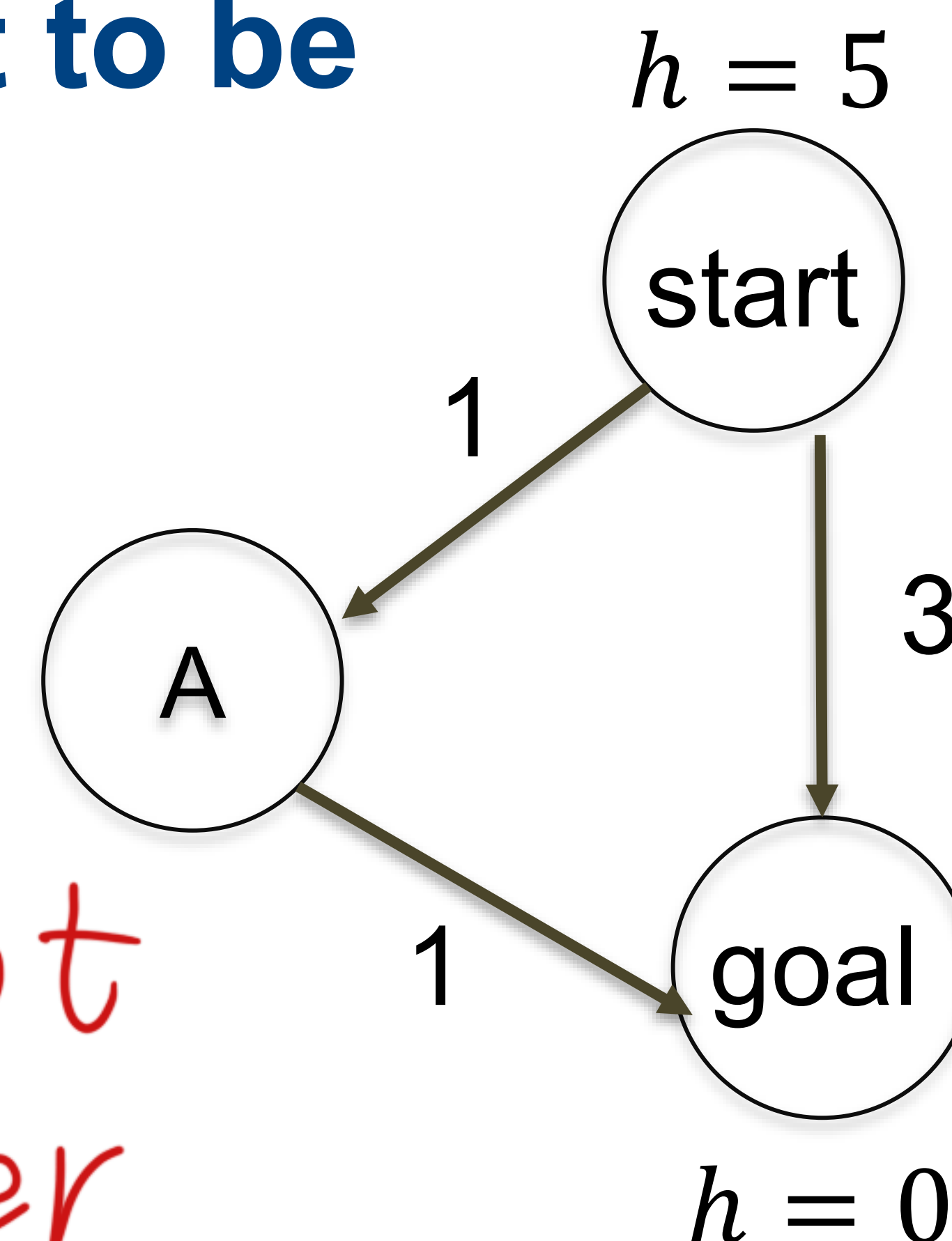


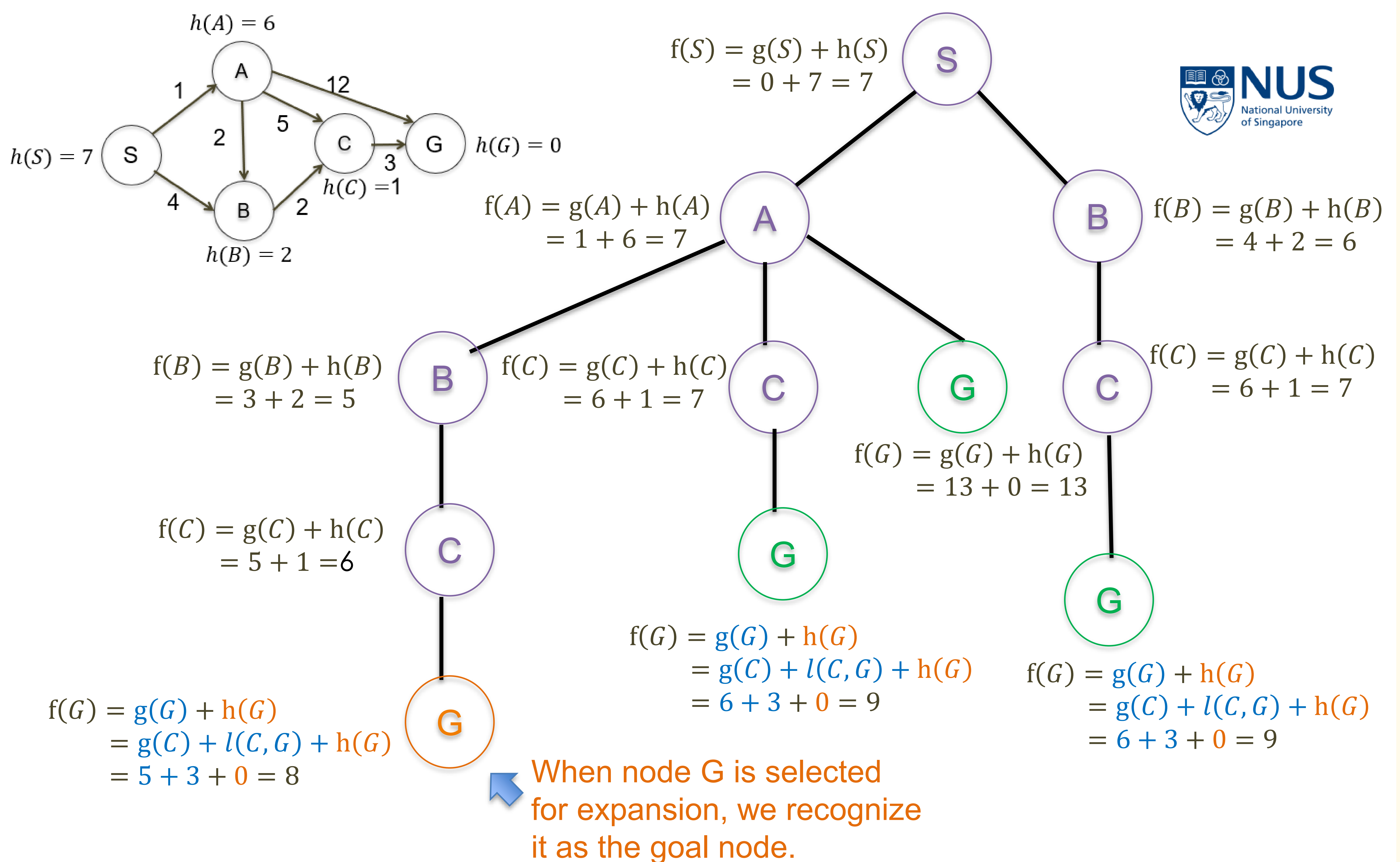
We only perform the goal test *when expanding a node*, and not *when adding it to the frontier*:

Because we can't be sure that we've found the *shortest path* to the goal until it's about to be expanded.

start $\xrightarrow{3}$ goal

So we do goal check when $h = 1$
 we intend to expand it, but not
 when we put it in the frontier

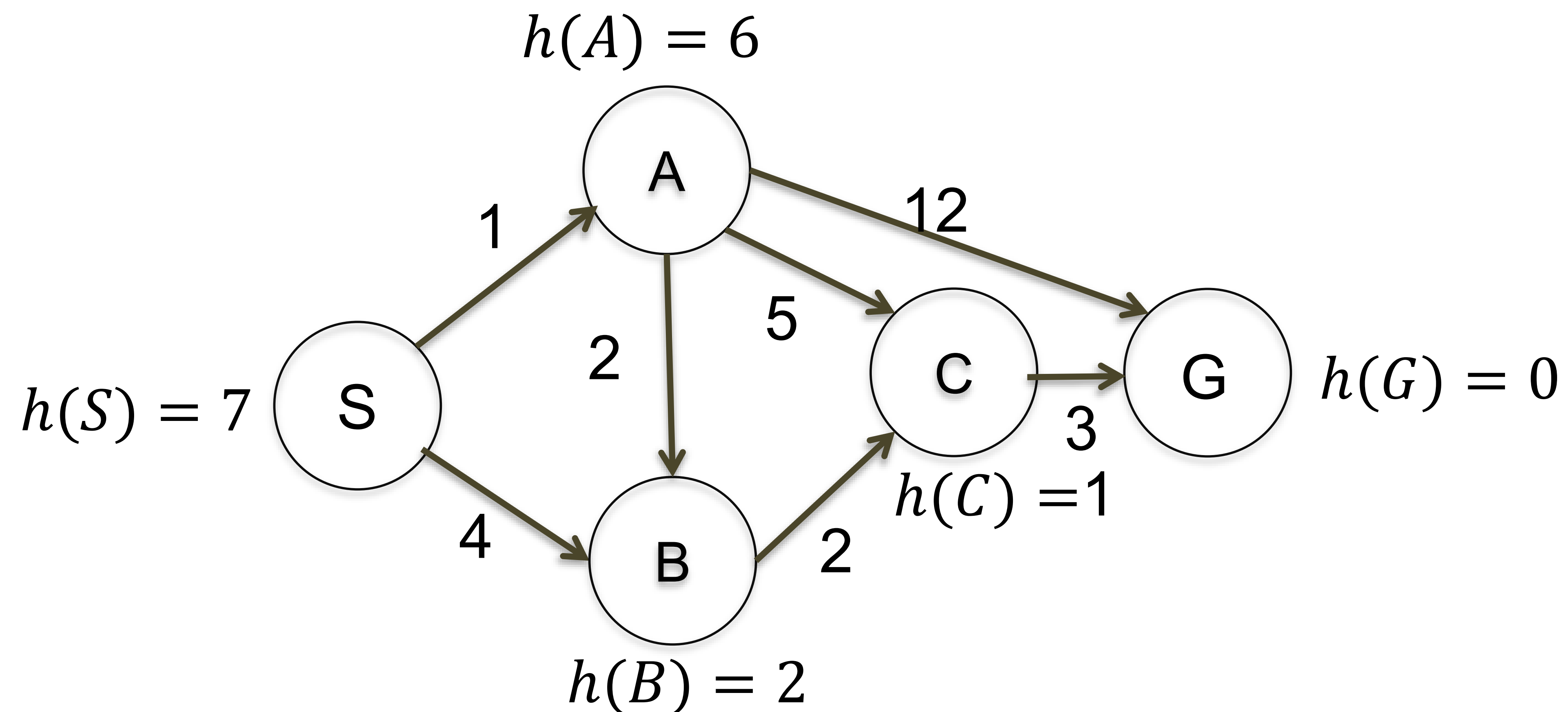




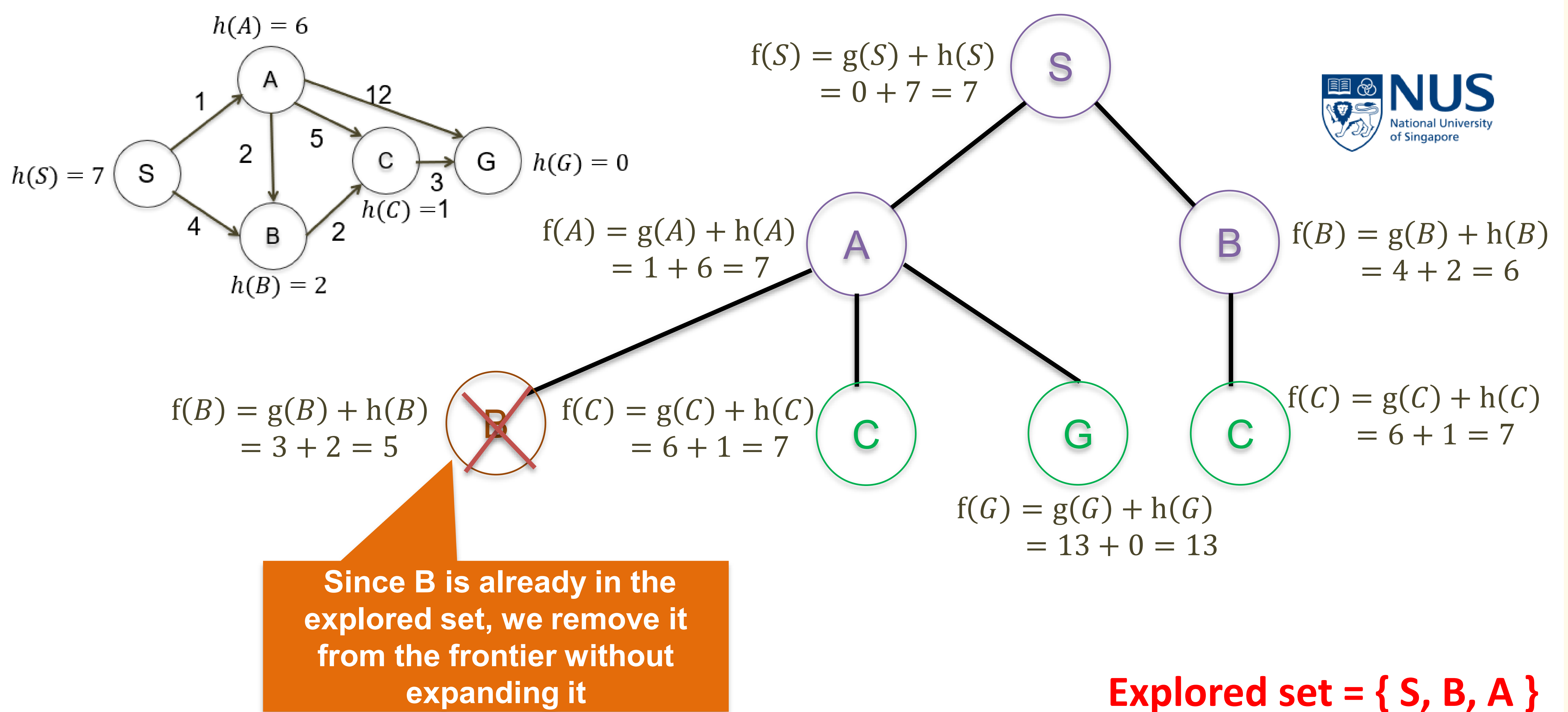
Optimal path: $S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$ with a total cost of 8

Class practice 2

Which path does A* search return using **graph search**? The starting node is S and goal node is G.



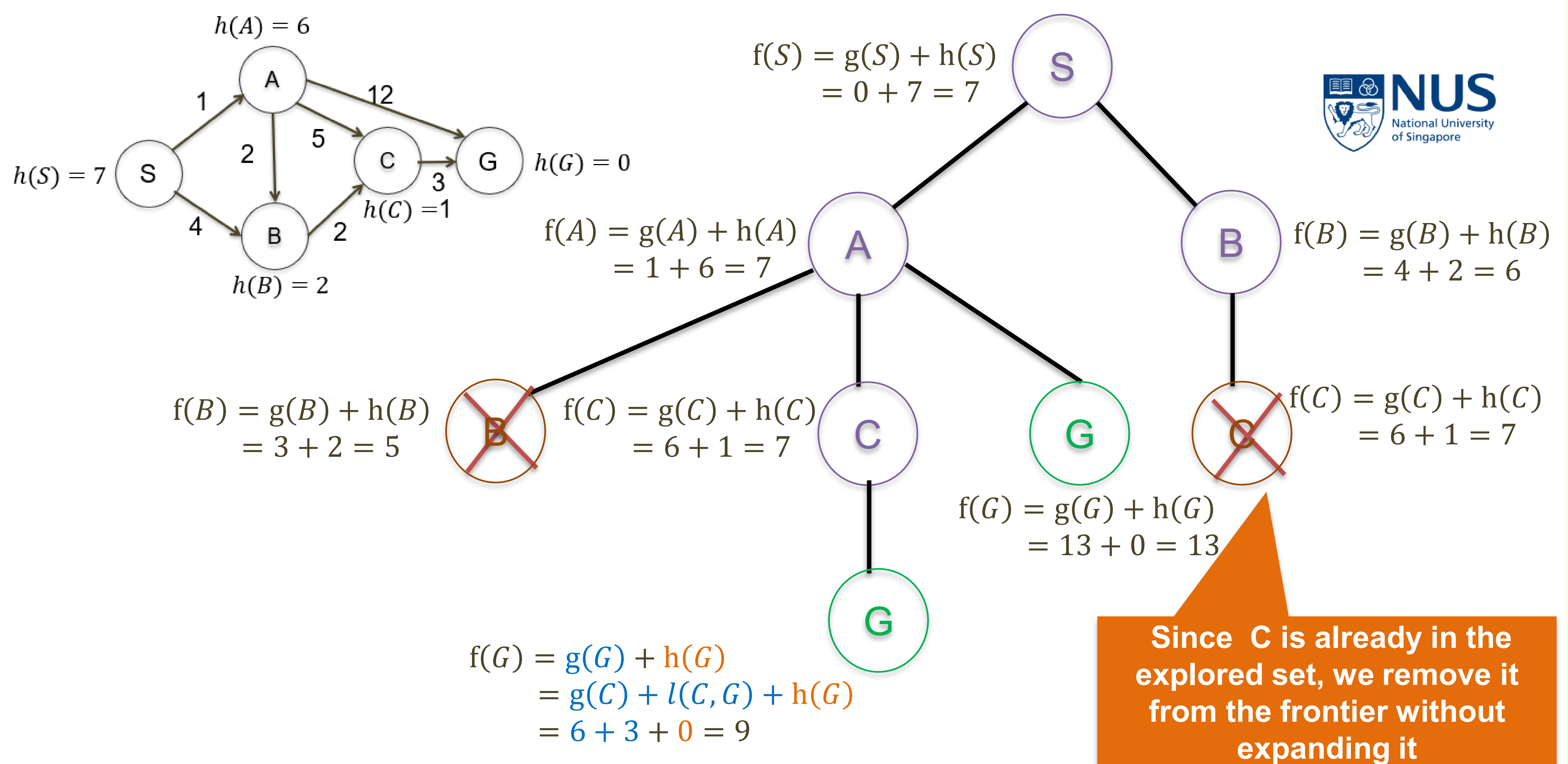
Tree search expands nodes without checking for duplicates, potentially revisiting the same state multiple times, whereas **graph search** keeps track of explored states to avoid revisiting them.



Lavender: Nodes that have been expanded (in the **explored set**)

Green: nodes currently on the **frontier**, waiting to be expanded.

Brown: nodes removed from the frontier without expansion because they had already been expanded earlier and are in the explored set.

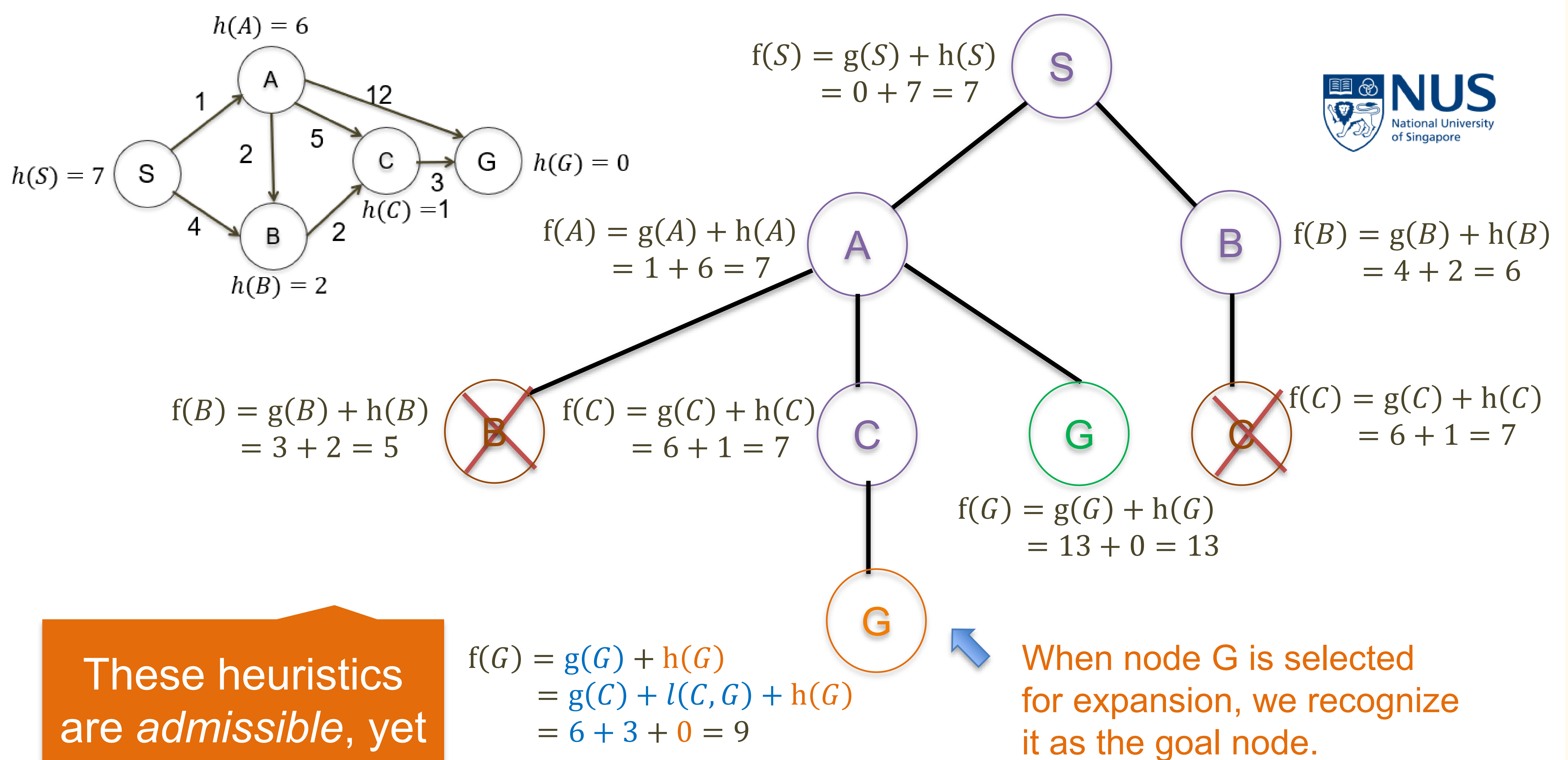


Lavender: Nodes that have been expanded (in the **explored set**)

Green: nodes currently on the **frontier**, waiting to be expanded.

Brown: nodes removed from the frontier without expansion because they had already been expanded earlier and are in the explored set.

Explored set = { S, B, A, C }



Explored set = { S, B, A, C }

Not optimal!

Resulting path: $S \rightarrow A \rightarrow C \rightarrow G$ with a total cost of 9

Optimal path: $S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$ with a total cost of 8

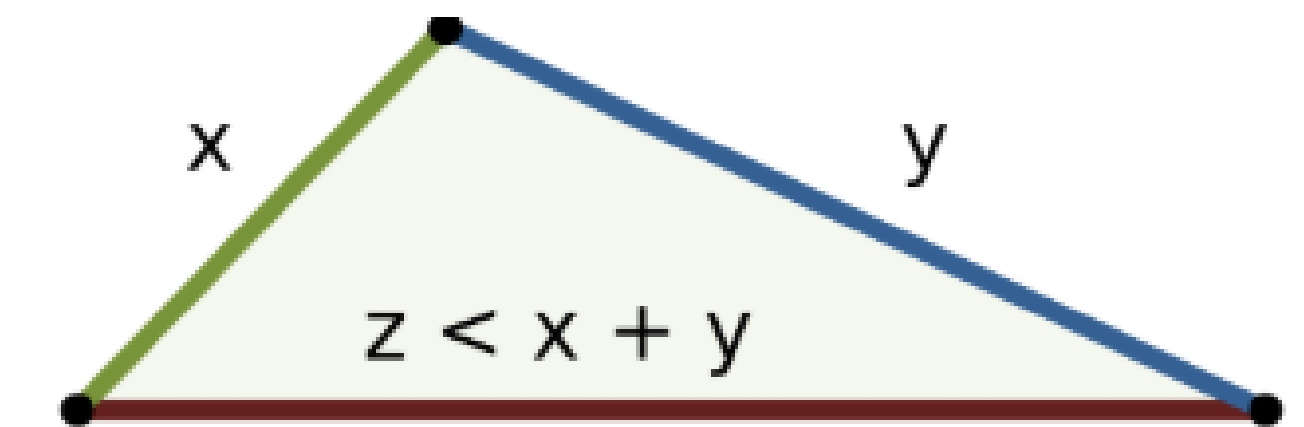
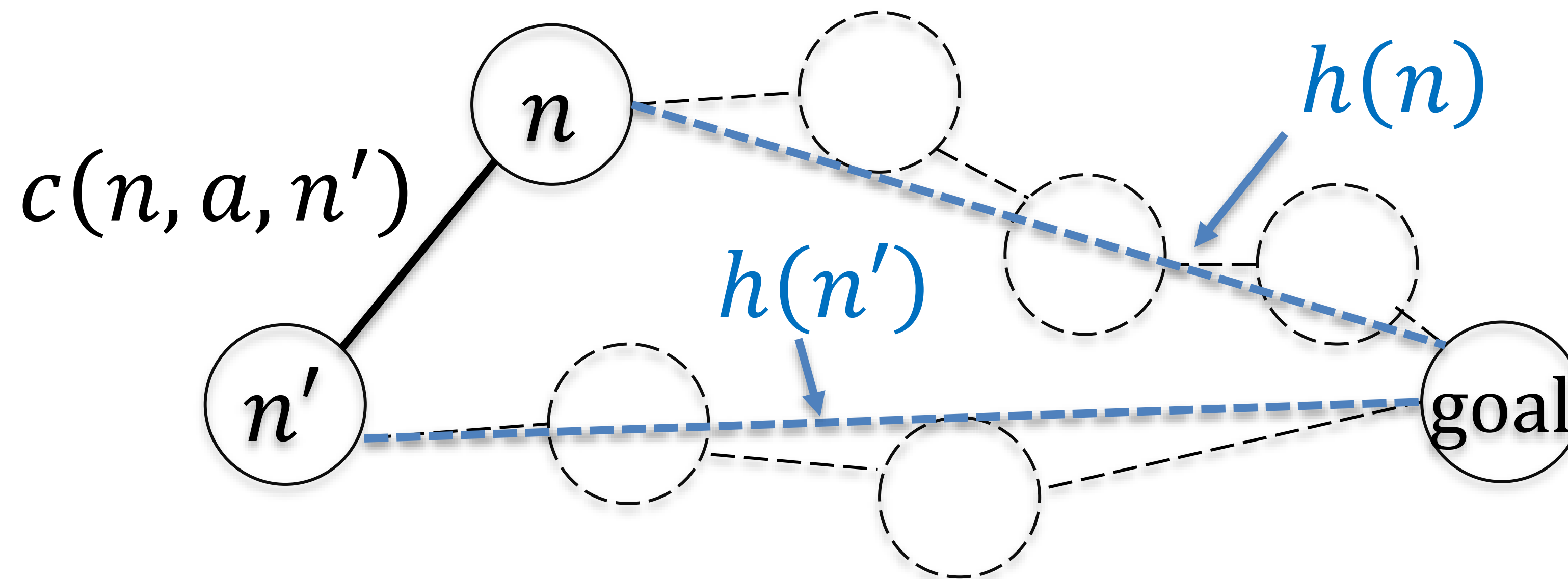
Consistent heuristics

A heuristic $h(n)$ is **consistent** (or monotonic) if for every node n , every successor n' of n generated by any action a :

$$h(n) \leq c(n, a, n') + h(n')$$

$c(n, a, n')$ is the actual cost of applying action a to go from state n to n'

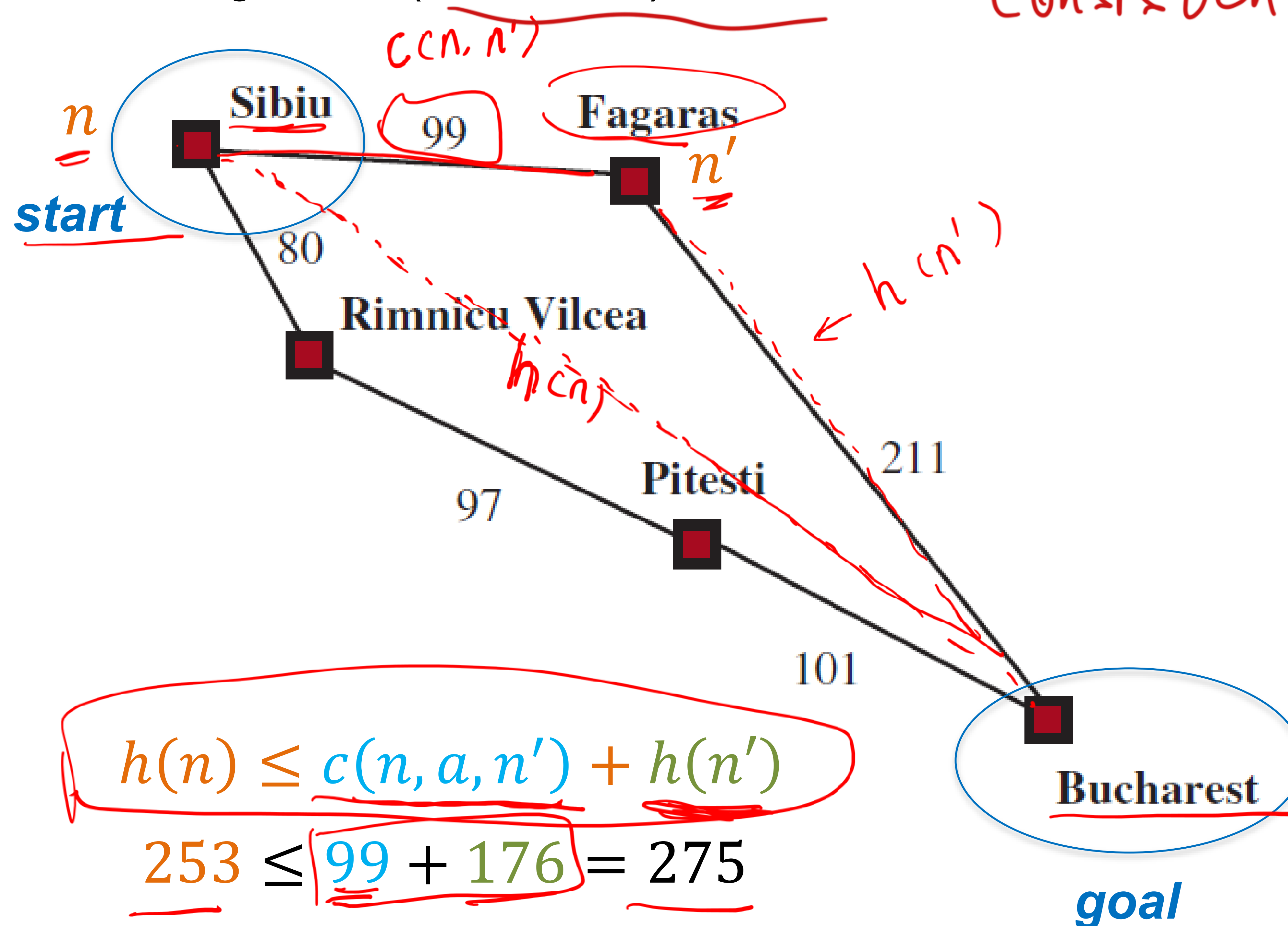
Consistency is a version of triangle inequality.



Consistency is a **stricter requirement than admissibility**.

Consistent heuristics: Example 1

Straight line (Euclidean) distance → Consistent and Admissible



$h(n)$ = Straight Line
Distances to Bucharest

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Consistent heuristics: Example 2

	10	9	8	7	6	5	4	3	2	1	B	<i>goal</i>
	11										1	
	12		10	9	8	7	6	5	4		2	
	13		11						5		3	
	14	13	12		10	9	8	7	6		4	
			13		11						5	
A	16	15	14		12	11	10	9	8	7	6	

Consistent

start

$$h(n) \leq c(n, a, n') + h(n')$$

$$11 \leq 6 + 5$$

Property of being consistent

If h is a consistent heuristic and all costs are non-zero, then $f(n)$ is non-decreasing along any path. (As you move toward the goal, the estimated total cost to reach it never decreases)

With **graph search**, if the heuristic is consistent, A^* never needs to re-expand a node. Once a node is removed from the frontier, the shortest path to that node has been *finalized* and will *remain optimal*.

Property of being consistent

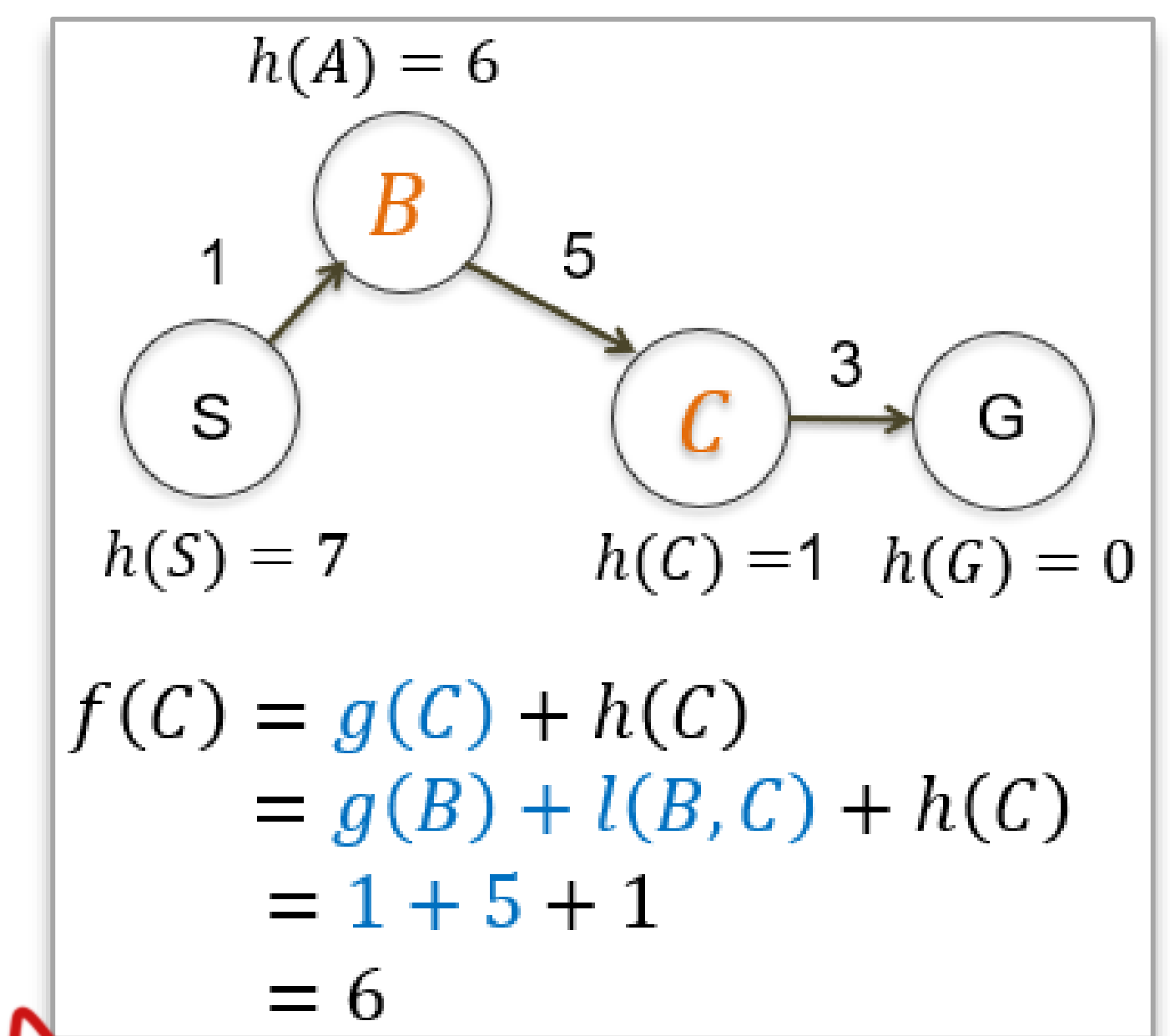
Claim:

If h is a consistent heuristic and all costs are non-zero, then $f(n)$ is **non-decreasing along any path**. (As you move toward the goal, the estimated total cost to reach it never decreases)

Proof:

Let n' be a successor of n via action a . Then

$$\begin{aligned} f(n') &= g(n') + h(n') \\ &= g(n) + c(n, a, n') + h(n') \end{aligned}$$



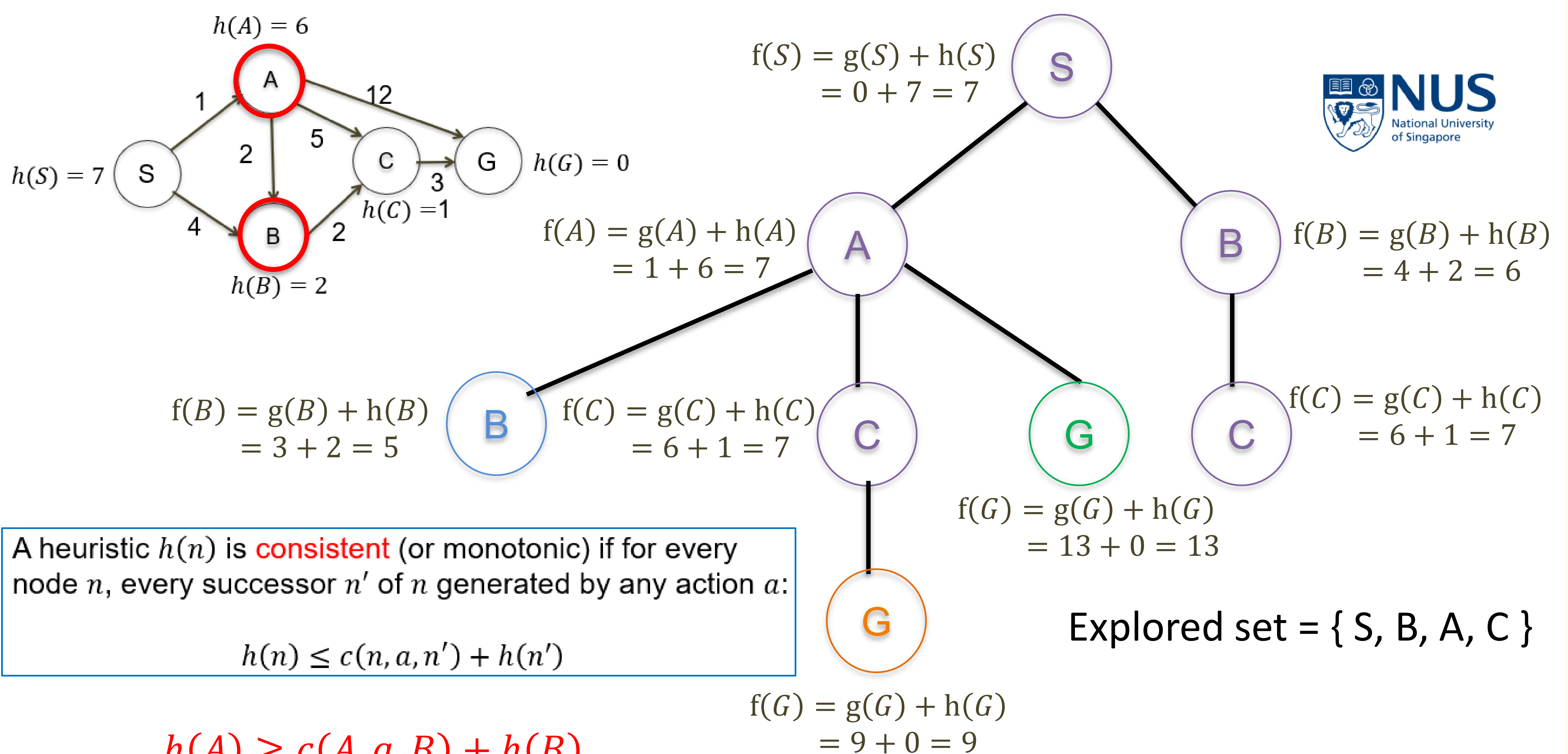
Since h is consistent, we have:

$$h(n) \leq c(n, a, n') + h(n')$$

So:

$$f(n') = g(n) + c(n, a, n') + h(n') \geq g(n) + h(n) = f(n)$$

This implies that when a node is explored, the shortest path to it has already been found, and any future path to that node will not be shorter.



A heuristic $h(n)$ is **consistent** (or monotonic) if for every node n , every successor n' of n generated by any action a :

$$h(n) \leq c(n, a, n') + h(n')$$

$$h(A) \geq c(A, a, B) + h(B)$$

$$6 \geq 2 + 2$$

Not consistent!

Resulting path: $S \rightarrow A \rightarrow C \rightarrow G$ with a total cost of 9

Optimal path: $S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$ with a total cost of 8

A* search properties

Optimality

- **Tree search** version of A* is optimal if the heuristic is *admissible*.
- **Graph search** version of A* is optimal if the heuristic is *consistent*.

Completeness

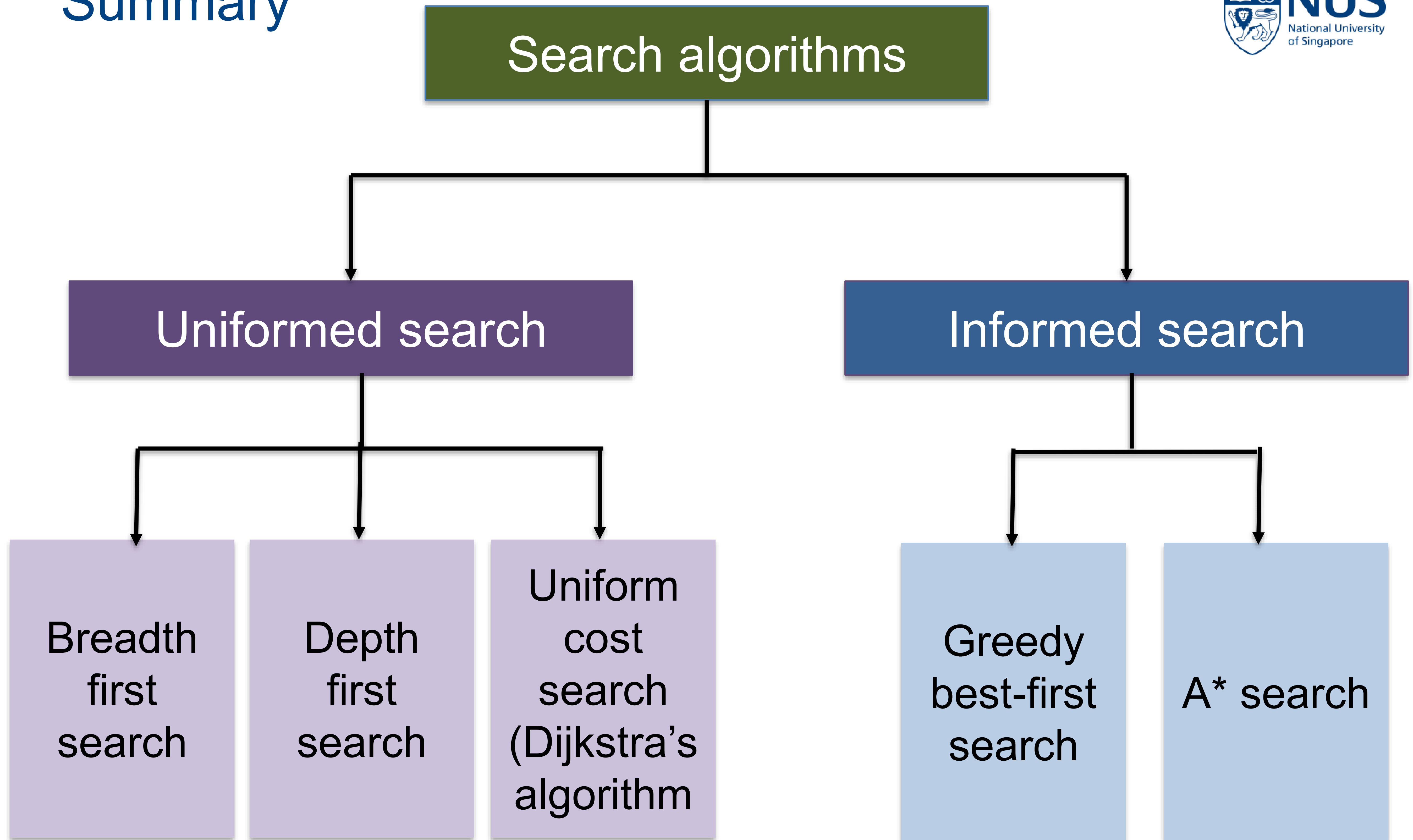
- A* search is generally complete (provided finite branching factor).

Consistency is a **stricter requirement** than admissibility.

Note: These properties of A* — optimality and completeness — **hold only when all edge costs are non-negative**. If negative edge costs exist, A* may fail to find the optimal path or even a solution at all.

Summary

- A* combines the cost so far (from the start) and an estimated cost to the goal. *← Dijkstra's*
← greedy BFS
- A* is optimal with admissible (tree search) / consistent (graph search) heuristics
- Heuristic design is key
- When the heuristic values are all zero, A* search behaves exactly like Dijkstra's algorithm.



Summary: Uninformed search & informed search

Uninformed search (BFS, DFS, Dijkstra's)

- Has no additional knowledge beyond what is given in the problem definition.
- Generate successor states and check whether each is a goal.
- As a result, the search proceeds blindly—systematically exploring the space until the goal is found.
- Search methods are distinguished by the order in which the nodes are expanded.

Informed search (greedy best-first, A*)

- Uses additional knowledge (a heuristic) to estimate how close a state is to the goal.
- The heuristic guides the search, often reducing the number of nodes explored and improving efficiency.

Summary

Feature	Uninformed search	Informed search
Search Algorithms	BFS, DFS, UCS (Dijkstra's algorithm)	Greedy best-first search, A* search
Additional knowledge available?	No (explores blindly, only uses problem definition, i.e., start, actions, goal)	Yes (uses domain-specific knowledge/heuristics to guide search)
Complete?	BFS & UCS (Dijkstra's algorithm): Yes DFS: No (can get stuck in infinite paths)	Greedy best-first: No (can get stuck in local optima) A*: Generally yes (finite search space and non-negative edge cost)
Optimal?	BFS: Yes (if all step costs are equal/unweighted graph) DFS: No (finds any path, not necessarily shortest) UCS (Dijkstra's algorithm): Yes (finds optimal path in weighted graphs with non-negative costs)	Greedy best-first: No (may find suboptimal path instead) A*: Yes (with admissible & consistent heuristic)
Efficiency	Generally less efficient for large search spaces, as it explores broadly/deeply without guidance. May explore many irrelevant paths	Generally more efficient for large search spaces, as heuristics guide it towards the goal.
When to use	Small or simple problems No heuristic available	Larger or more complex problems When good heuristic is available

STRIPS: Example

Search Concept (What they know)	STRIPS Concept (What you're introducing)	Example (Coffee Robot)
State	A state is a collection of logical propositions that are true.	{Has(Coffee), On(Mug, Table)}
Goal Test	The goal is a set of conditions that must be true.	{Has(Coffee)}
Actions	An action is an operator with: - Preconditions: Logical facts that must be true before you can perform the action. - Effects: How the action changes the state. It adds and removes facts.	Action: Pour(CoffeePot, Mug) Pre: Has(CoffeePot), On(Mug, Table) Effect: ~Has(CoffeePot), Has(Coffee)
Path Cost	Often just the number of actions (plan length).	Cost of [Grasp(Mug), Pour(...)] = 2
Successor Function	Applying an action. For a given state, check an action's preconditions. If they are all true, the new state is: (Old State - Effects.Remove) + Effects.Add	

With STRIPS modeling, we can represent the planning problem as a graph, where nodes are states and edges are actions, and then apply the search algorithms we've learned to find a solution.

STRIPS: Example

How this creates a search problem

Use BFS to find a plan:

State: {Has(CoffeePot), On(Mug, Table)} (*Initial State*)

Action: Apply Pour(CoffeePot, Mug)

- Check Preconditions: True.

Calculate New State:

- Remove: Has(CoffeePot)
- Add: Has(Coffee)

New State: {Has(Coffee), On(Mug, Table)} *GOAL!*

The Plan (Path): [Pour(CoffeePot, Mug)]

Note: BFS systematically explores all possible action sequences level by level, guaranteeing it will find the shortest plan (fewest actions) if one exists.

Demo: Refer to Lecture4_STRIPS_Example.py

Reference: <https://ai.stanford.edu/~nilsson/OnlinePubs-Nils/PublishedPapers/strips.pdf>
<https://www.geeksforgeeks.org/artificial-intelligence/strips-in-ai/>